Teach Module 42: Kubernetes and OpenShift Architecture

This guide was created from the chat session for Module 42. The course document was used only to identify the module topic. The teaching content below is original practice-oriented instruction.

Module Topic

Kubernetes and OpenShift Architecture

Kubernetes is a platform for running containerized applications across a cluster of machines. Instead of manually choosing servers, starting processes, restarting failed apps, and wiring networking by hand, you describe the desired state of your system.

A simple desired state might be:

```
Run 3 copies of my Java API.
Expose it with a stable service name.
Restart it if it crashes.
Roll out new versions safely.
```

Kubernetes continuously works to make the real cluster match that desired state.

OpenShift is Red Hat's enterprise platform built on Kubernetes. It includes Kubernetes, then adds developer workflows, stronger default security, integrated routing, image build features, web console tools, and enterprise operations features.

For a Java software engineer, this module is about understanding where your Spring Boot application lives after it becomes a container image.

```
Java code -> JAR -> container image -> Deployment -> Pod -> Service -> Route/Ingress -> user request
```

1. The Big Picture

When you deploy a Java application to Kubernetes, you usually do not deploy source code directly. You deploy a container image.

The usual flow is:

```
Write Spring Boot app
Build a JAR file
Create a container image
Push image to a registry
Deploy image to Kubernetes or OpenShift
Expose the app through a Service and Route/Ingress
Monitor, scale, update, and troubleshoot it
```

Kubernetes is not just a container runner. It is an orchestration system.

That means it handles concerns such as:

- where containers should run
- how many copies should exist
- what happens when a container crashes
- how services discover each other
- how configuration is injected
- how updates and rollbacks happen
- how traffic reaches the application

2. Cluster Architecture

A Kubernetes cluster has two major areas:

```
Control Plane
Worker Nodes
```

The **control plane** is the brain of the cluster. It stores cluster state, accepts requests, schedules workloads, and runs controllers.

The **worker nodes** are the machines where your application containers actually run.

Simple architecture:

```
Developer / CI/CD
      |
      v
Kubernetes API Server
      |
      v
Control Plane
      |
      v
Worker Node
      |
      v
Pod
      |
      v
Container
      |
      v
Java App
```

You interact with the cluster through tools such as:

```
kubectl
oc
OpenShift Console
CI/CD pipeline
GitOps controller
```

3. Control Plane Components

The control plane contains the main decision-making components.

API Server

The API Server is the front door of Kubernetes.

Every request goes through it:

```
kubectl apply -f deployment.yaml
kubectl get pods
oc expose service hello-api
CI/CD deployment request
```

If you create, update, or inspect a Kubernetes object, you are talking to the API Server.

Scheduler

The Scheduler decides where a new pod should run.

It considers things like:

- available CPU and memory
- node health
- constraints
- taints and tolerations
- affinity rules

For example, if you request 3 replicas of a Java API, the Scheduler decides which worker nodes should host those pods.

Controller Manager

Controllers watch the cluster and fix drift.

Example:

```
Desired state: 3 pods
Actual state: 2 pods
Controller action: create 1 more pod
```

This is why Kubernetes feels self-healing. It is constantly comparing what you asked for with what is actually running.

etcd

`etcd` is the cluster's key-value database.

It stores Kubernetes state, such as:

- deployments
- pods
- services
- secrets

- config maps
- namespaces
- node information

If the API Server is the front door, `etcd` is the source of truth behind it.

4. Worker Node Components

Worker nodes run the actual workloads.

kubelet

The `kubelet` is the node agent.

It receives instructions from the control plane and makes sure the requested containers are running on that node.

Container Runtime

The container runtime runs containers.

Common runtime:

```
containerd
```

Older tutorials may mention Docker as the runtime, but modern Kubernetes commonly uses containerd underneath.

kube-proxy

`kube-proxy` helps with service networking. It makes it possible for traffic sent to a Service to reach the correct pods.

5. Pods

A **Pod** is the smallest deployable unit in Kubernetes.

Usually, one pod contains one application container:

```
Pod
  |
  +-- Container: spring-boot-api
```

Sometimes a pod contains multiple containers, but for most Java backend services, one app container per pod is the normal starting point.

Important idea:

```
Pods are temporary.
```

They can be killed, replaced, rescheduled, or recreated. You should not design your application as if a pod is a permanent server.

6. Deployments

A **Deployment** manages replicas of your application.

Example desired state:

```
Application: order-service
Image: order-service:1.0.0
Replicas: 3
Port: 8080
```

Kubernetes uses the Deployment to create and maintain pods.

If one pod crashes, Kubernetes creates a replacement. If you update the image version, Kubernetes can perform a rolling update.

For Java engineers, a Deployment is usually the main object used to run a Spring Boot service.

7. Services

Pods are replaceable, so their IP addresses are not reliable for direct communication.

A **Service** gives a stable network name and address to a group of pods.

Example:

```
payment-service calls http://order-service:8080
```

The Service routes traffic to one of the healthy matching pods.

Without Services, one application would have to track constantly changing pod IP addresses. That would be painful and fragile.

8. ConfigMaps and Secrets

Java applications often need environment-specific configuration:

```
SPRING_PROFILES_ACTIVE
APP_MESSAGE
DB_HOST
API_TIMEOUT
FEATURE_FLAG_CHECKOUT
```

Kubernetes provides two common objects for this:

- `ConfigMap` for non-sensitive configuration
- `Secret` for sensitive configuration

Use `ConfigMap` for values like:

```
APP_MESSAGE=Hello from Kubernetes
SPRING_PROFILES_ACTIVE=dev
```

Use `Secret` for values like:

```
DB_PASSWORD
API_TOKEN
PRIVATE_KEY
```

In a Spring Boot app, these values are often injected as environment variables.

9. Ingress and OpenShift Routes

A Service gives stable networking inside the cluster.

For users outside the cluster, you need an external entry point.

In Kubernetes, this is commonly handled with:

```
Ingress
```

In OpenShift, this is commonly handled with:

```
Route
```

Example:

```
Browser
  |
  v
https://orders.company.com
  |
  v
Route or Ingress
  |
  v
Service
  |
  v
Pod
  |
  v
Spring Boot app
```

10. OpenShift Architecture

OpenShift includes Kubernetes, then adds a more complete enterprise developer platform.

OpenShift commonly adds:

- web console
- projects
- routes
- integrated developer workflows
- image build features

- image registry integration
- stronger default security
- OperatorHub
- enterprise authentication and authorization patterns

Useful comparison:

```
Kubernetes = orchestration foundation
OpenShift = Kubernetes + enterprise developer platform
```

Kubernetes vs OpenShift

Concept	Kubernetes	OpenShift
CLI	`kubectl`	`oc`
Isolation unit	Namespace	Project
External access	Ingress	Route
Platform style	Core orchestration	Enterprise application platform
Security defaults	Flexible	More restrictive by default
Developer UI	Optional	Built-in web console

11. Practice Exercises

Exercise 1: Draw the Architecture

Draw this flow:

```
Developer
kubectl / oc CLI
API Server
Scheduler
Worker Node
Pod
Container
Service
Route / Ingress
```

Then explain what happens when a Java app is deployed.

Goal: understand the moving parts before focusing on commands.

Exercise 2: Containerize a Simple Spring Boot App

Create a simple Spring Boot REST API with:

```
GET /hello
GET /version
```

Expected output:

```
Hello from Kubernetes
v1
```

Build and run it as a container:

```
docker build -t hello-java-api:1.0 .
docker run -p 8080:8080 hello-java-api:1.0
```

Goal: understand that Kubernetes runs container images, not raw Java source code.

Exercise 3: Create a Deployment

Create a `deployment.yaml` with:

```
replicas: 2
containerPort: 8080
image: hello-java-api:1.0
```

Apply it:

```
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get pods
```

Goal: understand how Deployments create and manage Pods.

Exercise 4: Expose the App with a Service

Create a `service.yaml` for the app.

Apply it:

```
kubectl apply -f service.yaml
kubectl get services
```

Test with port forwarding:

```
kubectl port-forward service/hello-java-api-service 8080:8080
curl http://localhost:8080/hello
```

Goal: understand why Services exist when Pods are temporary.

Exercise 5: Scale the Application

Scale the app:

```
kubectl scale deployment hello-java-api --replicas=4
```

```
kubectl get pods
```

Then scale it back:

```
kubectl scale deployment hello-java-api --replicas=2
```

Goal: understand horizontal scaling.

Exercise 6: Simulate Self-Healing

Delete one pod:

```
kubectl delete pod <pod-name>
kubectl get pods
```

Watch Kubernetes create a replacement.

Goal: understand desired state and self-healing.

Exercise 7: Add a ConfigMap

Create a ConfigMap:

```
APP_MESSAGE=Hello from Kubernetes ConfigMap
```

Inject it into the Spring Boot application as an environment variable.

Goal: understand externalized configuration.

Exercise 8: Add a Secret

Create a fake database password:

```
kubectl create secret generic db-secret --from-literal=DB_PASSWORD=training123
```

Inject it into the app as an environment variable.

Goal: understand the difference between ConfigMaps and Secrets.

Exercise 9: Add Health Checks

Add Spring Boot Actuator and expose:

```
/actuator/health
```

Configure:

```
livenessProbe
readinessProbe
```

Goal: understand how Kubernetes knows whether your app is alive and ready for traffic.

Exercise 10: Rolling Update

Deploy version `1.0`, then build and deploy version `2.0`.

```
kubectl set image deployment/hello-java-api hello-java-api=hello-java-api:2.0
kubectl rollout status deployment/hello-java-api
kubectl rollout history deployment/hello-java-api
```

Goal: understand rolling deployments.

Exercise 11: Rollback

Rollback to the previous version:

```
kubectl rollout undo deployment/hello-java-api
kubectl rollout status deployment/hello-java-api
```

Goal: understand safer production releases.

Exercise 12: OpenShift Route Practice

If using OpenShift:

```
oc new-project module42-lab
oc apply -f deployment.yaml
oc apply -f service.yaml
oc expose service hello-java-api-service
oc get routes
```

Goal: understand how OpenShift exposes applications externally.

Exercise 13: Kubernetes and OpenShift Comparison

Create a two-column table comparing:

```
Namespace vs Project
Ingress vs Route
kubectl vs oc
Basic orchestration vs developer platform
Flexible security vs stricter defaults
```

Goal: understand what OpenShift adds on top of Kubernetes.

Exercise 14: Troubleshooting Drill

Break something intentionally:

- use a wrong image name
- use a wrong port
- remove a required environment variable
- use an incorrect health check path

Then troubleshoot:

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events
kubectl get deployment hello-java-api
kubectl get service hello-java-api-service
```

Goal: learn how to diagnose failed deployments.

12. Full Lab: Deploy and Manage a Java Spring Boot App

Lab Goal

Containerize a simple Java application, deploy it to Kubernetes or OpenShift, expose it, scale it, update it, roll it back, and troubleshoot it.

Prerequisites

Use one of these environments:

```
Docker Desktop with Kubernetes enabled
Minikube
OpenShift Local / CRC
Existing OpenShift cluster
```

Install:

```
Java 17+
Maven
Docker or Podman
kubectl
oc, if using OpenShift
```

Part 1: Create a Simple Spring Boot App

Create a controller:

```java
import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @Value("${APP_MESSAGE:Hello from local Java}")
    private String message;

    @GetMapping("/hello")
```

```
    public String hello() {
        return message;
    }

    @GetMapping("/version")
    public String version() {
        return "v1";
    }
}
```

Add Spring Boot Actuator:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Run locally:

```
mvn spring-boot:run
```

Test:

```
curl http://localhost:8080/hello
curl http://localhost:8080/version
curl http://localhost:8080/actuator/health
```

Part 2: Create a Dockerfile

Create `Dockerfile` :

```
FROM eclipse-temurin:17-jdk-alpine
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Build the app:

```
mvn clean package
docker build -t hello-java-api:1.0 .
```

Run it:

```
docker run -p 8080:8080 hello-java-api:1.0
```

Part 3: Create Kubernetes Deployment

Create `deployment.yaml`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-java-api
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-java-api
  template:
    metadata:
      labels:
        app: hello-java-api
    spec:
      containers:
        - name: hello-java-api
          image: hello-java-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
```

Apply:

```
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get pods
```

Part 4: Expose with a Service

Create `service.yaml`:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: hello-java-api-service
spec:
  selector:
    app: hello-java-api
  ports:
    - port: 8080
      targetPort: 8080
  type: ClusterIP
```

Apply:

```
kubectl apply -f service.yaml
kubectl get svc
```

Test:

```
kubectl port-forward service/hello-java-api-service 8080:8080
curl http://localhost:8080/hello
```

Part 5: Add a ConfigMap

Create `configmap.yaml`:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: hello-config
data:
  APP_MESSAGE: "Hello from Kubernetes ConfigMap"
```

Apply:

```
kubectl apply -f configmap.yaml
```

Update the Deployment container section:

```yaml
envFrom:
  - configMapRef:
      name: hello-config
```

Reapply and restart:

```
kubectl apply -f deployment.yaml
kubectl rollout restart deployment hello-java-api
```

Expected result:

```
Hello from Kubernetes ConfigMap
```

Part 6: Add Health Probes

Add probes to the container definition:

```yaml
livenessProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  initialDelaySeconds: 20
  periodSeconds: 10

readinessProbe:
  httpGet:
```

```
    path: /actuator/health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

Apply and inspect:

```
kubectl apply -f deployment.yaml
kubectl describe pod <pod-name>
```

Part 7: Scale the App

Scale to 4 replicas:

```
kubectl scale deployment hello-java-api --replicas=4
kubectl get pods
```

Scale back to 2:

```
kubectl scale deployment hello-java-api --replicas=2
```

Part 8: Self-Healing Test

Delete a pod:

```
kubectl delete pod <pod-name>
kubectl get pods
```

Kubernetes should create a replacement pod automatically.

Part 9: Rolling Update

Change the `/version` endpoint:

```
return "v2";
```

Rebuild:

```
mvn clean package
docker build -t hello-java-api:2.0 .
```

Update the Deployment:

```
kubectl set image deployment/hello-java-api hello-java-api=hello-java-api:2.0
kubectl rollout status deployment/hello-java-api
```

Test:

```
curl http://localhost:8080/version
```

Expected:

```
v2
```

Part 10: Rollback

Rollback:

```
kubectl rollout undo deployment/hello-java-api
kubectl rollout status deployment/hello-java-api
```

Check the version again:

```
curl http://localhost:8080/version
```

13. OpenShift Optional Lab

If using OpenShift:

```
oc new-project module42-lab
oc apply -f deployment.yaml
oc apply -f service.yaml
oc expose service hello-java-api-service
oc get routes
```

Then test the route URL:

```
curl http://<route-url>/hello
```

OpenShift-specific items to observe:

- project instead of plain namespace workflow
- route instead of manually configured ingress
- OpenShift web console view
- stricter security defaults
- `oc` commands that extend `kubectl`

14. Lab Deliverables

Submit evidence of:

```
1. Running pods
2. Running deployment
3. Running service
4. Output from /hello
5. Output from /version
```

```
6. Scaling from 2 to 4 replicas
7. Self-healing after deleting a pod
8. Rolling update from v1 to v2
9. Rollback from v2 to v1
10. OpenShift route, if using OpenShift
```

Also write short explanations of:

- Pod
- Deployment
- Service
- ConfigMap
- Secret
- Ingress
- Route
- Control plane
- Worker node

15. Troubleshooting Commands

Use these commands often:

```
kubectl get pods
kubectl get deployments
kubectl get services
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events
kubectl rollout status deployment/hello-java-api
kubectl rollout history deployment/hello-java-api
```

OpenShift equivalents:

```
oc get pods
oc get deployments
oc get services
oc get routes
oc describe pod <pod-name>
oc logs <pod-name>
oc get events
```

16. Knowledge Check

Answer these without looking:

1. What is the difference between a Pod and a Deployment?
2. Why should one service not call another pod directly by IP address?
3. What does the Kubernetes Scheduler do?
4. What does the Controller Manager do?
5. What is stored in `etcd`?
6. Why are ConfigMaps and Secrets separate?

7. What is the difference between liveness and readiness?
8. What does OpenShift add on top of Kubernetes?
9. What is the difference between an OpenShift Project and a Kubernetes Namespace?
10. What happens when you delete a pod managed by a Deployment?

17. Final Recap

Kubernetes gives you a way to run Java applications reliably across a cluster.

OpenShift builds on Kubernetes and adds enterprise developer workflows, routes, stronger default security, a web console, and platform-level tools.

The most important Module 42 flow is:

```
Java app
  -> JAR
  -> container image
  -> Deployment
  -> Pod
  -> Service
  -> Route or Ingress
  -> user traffic
```

If you can explain that flow clearly and complete the lab, you have the foundation needed for Kubernetes and OpenShift application deployment.